

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vtu29748@veltech.edu.in

Roll no: vtu29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 8_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 31

Section 1 : Coding

1. Problem Statement

You are tasked with finding the shortest communication paths from a source router to all other routers in a network using Dijkstra's algorithm.

The network is represented as a graph, where routers are nodes and communication links between them are edges with weights representing the time or cost of communication. A value of 0 in the adjacency matrix indicates no direct link between two routers.

Example

Explanation:

The minimum distance from 0 to 1 = 4. Path: 0 1

The minimum distance from 0 to 2 = 12. Path: 0 1 2

The minimum distance from 0 to 3 = 19. Path: 0 1 2 3

The minimum distance from 0 to 4 = 21. Path: 0 7 6 5 4

The minimum distance from 0 to 5 = 11. Path: 0 7 6 5

The minimum distance from 0 to 6 = 9. Path: 0 7 6

The minimum distance from 0 to 7 = 8. Path: 0 7

The minimum distance from 0 to 8 = 14. Path: 0 1 2 8

Input Format

The first line contains an integer V , representing the number of vertices in the graph.

The next V lines each contain V space-separated integers, representing the adjacency matrix of the graph. A value of 0 indicates no direct edge between the vertices.

The last line contains an integer src , representing the source vertex.

Output Format

The first line of output prints: Vertex followed by a tab character followed by Distance from Source

The next V lines each print the vertex number followed by three tab characters, a space, and the shortest distance from the source. Vertices are printed in ascending order from 0 to $V-1$.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 1 3 1

1 0 4 2

3 4 0 5

1 2 5 0

1

Output: Vertex Distance from Source

0	1
1	0
2	4
3	2

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX 10
```

```
int main() {
```

```
    int V;
```

```
    scanf("%d", &V);
```

```
    int graph[MAX][MAX];
```

```
    for (int i = 0; i < V; i++)
```

```
        for (int j = 0; j < V; j++)
```

```
            scanf("%d", &graph[i][j]);
```

```
    int src;
```

```
    scanf("%d", &src);
```

```
    int dist[MAX], visited[MAX];
```

```
    for (int i = 0; i < V; i++) {
```

```
        dist[i] = INT_MAX;
```

```
        visited[i] = 0;
```

```
    }
```

```
    dist[src] = 0;
```

```
    for (int count = 0; count < V - 1; count++) {
```

```
        int min = INT_MAX, u = -1;
```

```
        for (int i = 0; i < V; i++) {
```

```
            if (!visited[i] && dist[i] <= min) {
```

```
                min = dist[i];
```

```
                u = i;
```

```
            }
```

```

    }
    if (u == -1) break;

    visited[u] = 1;

    for (int v = 0; v < V; v++) {
        if (!visited[v] && graph[u][v] != 0 && dist[u] != INT_MAX &&
            dist[u] + graph[u][v] < dist[v]) {
            dist[v] = dist[u] + graph[u][v];
        }
    }
}

printf("Vertex \tDistance from Source\n");
for (int i = 0; i < V; i++) {
    printf("%d\t\t\t %d\n", i, dist[i]);
}

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Given a network of cities represented as an unweighted graph, find the shortest path between a source city and a destination city in terms of the minimum number of roads to be traversed. Implement a program that utilizes the Dijkstra algorithm to solve this problem.

The program should take the city network, source city, and destination city as inputs, and provide the shortest path length and the sequence of cities to be visited along the path as outputs.

If no path exists between the source and destination, print 'No path found'.
 Note: Since the graph is unweighted, all edges have an implicit weight of 1.

Input Format

The first line contains an integer n , representing the number of cities in the network.

The second line contains an integer m , representing the number of roads in the city network.

The following m lines contain pairs of integers u and v , representing a road between cities u and v .

The next line contains an integer s , representing the source city.

The last line contains an integer d , representing the destination city.

Output Format

The first line prints Shortest path length: followed by the number of edges in the shortest path.

The second line prints Path: followed by the city indices along the shortest path, each followed by a space.

If no path exists, print No path found.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8

10

0 1

0 3

1 2

3 4

3 7

4 5

4 6

4 7

5 6

6 7

0

7

Output: Shortest path length: 2
Path: 0 3 7

Answer

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int main() {  
    int n, m;  
    scanf("%d", &n);  
    scanf("%d", &m);
```

```
    int adj[MAX][MAX] = {0};
```

```
    for (int i = 0; i < m; i++) {  
        int u, v;  
        scanf("%d %d", &u, &v);  
        adj[u][v] = 1;  
        adj[v][u] = 1;  
    }
```

```
    int s, d;  
    scanf("%d", &s);  
    scanf("%d", &d);
```

```
    int queue[MAX], front = 0, rear = 0;  
    int visited[MAX] = {0}, parent[MAX];
```

```
    for (int i = 0; i < n; i++)  
        parent[i] = -1;
```

```
    queue[rear++] = s;  
    visited[s] = 1;
```

```
    while (front < rear) {  
        int u = queue[front++];
```

```
        if (u == d)  
            break;
```

```
        for (int v = 0; v < n; v++) {
```

```

        if (adj[u][v] && !visited[v]) {
            visited[v] = 1;
            parent[v] = u;
            queue[rear++] = v;
        }
    }
}

if (!visited[d]) {
    printf("No path found");
    return 0;
}

int path[MAX], len = 0;
int temp = d;

while (temp != -1) {
    path[len++] = temp;
    temp = parent[temp];
}

printf("Shortest path length: %d\n", len - 1);
printf("Path: ");

for (int i = len - 1; i >= 0; i--)
    printf("%d ", path[i]);

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Liam, a compiler designer, is optimizing task scheduling in a dependency-based system. He is working with a Directed Acyclic Graph (DAG), where each node represents a task, and a directed edge from one node to another signifies a dependency.

To schedule the tasks optimally, Liam must perform a topological sort

using Depth-First Search (DFS), sorting the tasks based on their departure times in descending order.

Your task is to help Liam implement this topological sorting algorithm.

Input Format

The first line contains an integer N, representing the number of vertices (tasks) in the DAG.

The second line contains an integer E, representing the number of directed edges (dependencies) in the graph.

The next E lines each contain two space-separated integers:

- src The starting vertex of a directed edge.
- dest The ending vertex of a directed edge.

Output Format

The output prints N space-separated integers in a single line, representing the 0-indexed vertices in descending order of their DFS departure times, each followed by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

6

5 2

5 0

4 0

4 1

2 3

3 1

Output: 5 4 2 3 1 0

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```



```
#define MAX 100
```

```
int adj[MAX][MAX];  
int visited[MAX];  
int stack[MAX];  
int top = -1;
```

```
void push(int v) {  
    stack[++top] = v;  
}
```

```
void dfs(int v, int n) {  
    visited[v] = 1;  
    for (int i = 0; i < n; i++) {  
        if (adj[v][i] && !visited[i]) {  
            dfs(i, n);  
        }  
    }  
    push(v);  
}
```

```
int main() {  
    int n, e;  
    if (scanf("%d", &n) != 1) return 0;  
    if (scanf("%d", &e) != 1) return 0;  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            adj[i][j] = 0;  
        }  
        visited[i] = 0;  
    }  
  
    for (int i = 0; i < e; i++) {  
        int u, v;  
        scanf("%d %d", &u, &v);  
        adj[u][v] = 1;  
    }  
  
    for (int i = 0; i < n; i++) {  
        if (!visited[i]) {
```

```

        dfs(i, n);
    }
}

while (top >= 0) {
    printf("%d ", stack[top--]);
}
printf("\n");

return 0;
}

```

Status : Partially correct

Marks : 1/10

4. Problem Statement:

You are given a directed acyclic graph (DAG) G with N vertices and M edges. From G , a new graph $G^{\wedge}$ is constructed using the same vertex set and the following rules:

If there exists an edge from vertex u to vertex v in G , then there is no edge from v to u in $G^{\wedge}$. If there exists an edge from u to v in G , and from v to w in G , then there exists an edge from u to w in $G^{\wedge}$. In $G^{\wedge}$, find the maximum possible size of a set S of vertices such that for every pair of vertices (u, v) in S , there exists a directed edge between them (either $u \rightarrow v$ or $v \rightarrow u$).

Example:

Input: $N=3, M=2$, edges: 1 2, 2 3

$G^{\wedge}$ contains edges: 1 2, 2 3, and 1 3 (by rule 2).

Set $S = \{1, 2, 3\}$: edge exists between every pair (1 2, 2 3, 1 3). This is valid with size 3.

Output: 3

Input Format

The first line contains two space-separated integers N and M , representing the number of vertices and edges in graph G .

The next M lines each contain two space-separated integers u and v ($1 \leq u, v \leq N$), representing a directed edge from vertex u to vertex v.

Output Format

Print a single integer representing the maximum possible size of set S.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 3 2

1 2

2 3

Output: 3

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAXN 101
```

```
int adj[MAXN][MAXN];
```

```
int dp[MAXN];
```

```
int n, m;
```

```
int solve(int u) {
```

```
    if (dp[u] != -1) {
```

```
        return dp[u];
```

```
    }
```

```
    int max_len = 0;
```

```
    for (int v = 1; v <= n; v++) {
```

```
        if (adj[u][v]) {
```

```
            int current = solve(v);
```

```
            if (current > max_len) {
```

```
                max_len = current;
```

```
            }
```

```
        }
```

```
    }
```

```

    return dp[u] = 1 + max_len;
}

int main() {
    if (scanf("%d %d", &n, &m) != 2) return 0;

    for (int i = 1; i <= n; i++) {
        dp[i] = -1;
        for (int j = 1; j <= n; j++) {
            adj[i][j] = 0;
        }
    }

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
    }

    int max_size = 0;
    for (int i = 1; i <= n; i++) {
        int current = solve(i);
        if (current > max_size) {
            max_size = current;
        }
    }

    printf("%d\n", max_size);

    return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

You are given a directed acyclic graph (DAG) represented as an adjacency list.

Your task is to perform a topological sort on the graph and print the vertices in topological order. The input graph is guaranteed to be a directed acyclic graph.

Input Format

The first line of input consists of an integer N, representing the number of vertices in the directed acyclic graph.

The second line consists of an integer E, representing the number of directed edges in the graph.

The following E lines consist of two space-separated integers, src and dest, representing a directed edge from vertex src to vertex dest.

Output Format

The output consists of the vertices in topological order separated by spaces.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6

6

5 2

5 0

4 0

4 1

2 3

3 1

Output: 4 5 0 2 3 1

Answer

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int main() {  
    int n, e;
```

```

scanf("%d", &n);
scanf("%d", &e);

int adj[MAX][MAX] = {0};
int indegree[MAX] = {0};

for (int i = 0; i < e; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    adj[u][v] = 1;
    indegree[v]++;
}

int visited[MAX] = {0};
for (int count = 0; count < n; count++) {
    int u = -1;
    for (int i = 0; i < n; i++) {
        if (!visited[i] && indegree[i] == 0) {
            u = i;
            break;
        }
    }

    if (u == -1) break;

    printf("%d ", u);
    visited[u] = 1;

    for (int v = 0; v < n; v++) {
        if (adj[u][v]) {
            indegree[v]--;
        }
    }
}

return 0;
}

```

Status : Wrong

Marks : 0/10